
ML Factor Investing

Complete Project Framework

Systematic Long-Short Equity Strategy on the S&P 500

with Gaussian Mixture Regime-Based Risk Management

Team of 3 • 5-Week Project

Project Goal. Build an ML-based cross-sectional equity strategy on the S&P 500 (long-short portfolio) with a Gaussian Mixture regime-detection overlay that modulates position sizing and risk exposure based on detected market conditions. The core engine is a sector-neutral long-short portfolio driven by ML return predictions. The regime model manages risk by scaling position sizes and tightening entry thresholds during turbulent periods. Evaluate with a rigorous walk-forward backtest including transaction costs. Primary benchmark: risk-adjusted performance (Sharpe ratio) versus the S&P 500 index. Methodology inspired by Gu, Kelly & Xiu (2020).

This document. The complete, self-contained technical framework for the project. It defines the strategy architecture, the ML pipeline, the data pipeline, the regime overlay, the backtest methodology, and the evaluation framework. Incorporates research findings from Gu, Kelly & Xiu (2020), the team's data pipeline checklist, and the stock prediction guide. Every design decision is justified.

Table of Contents

1. Strategy Architecture Overview
2. Why Long-Short with Regime Risk Management
3. The ML Prediction Engine (Person B)
4. The Data Pipeline (Person A)
5. The Regime Overlay (Person C)
6. Portfolio Construction
7. Walk-Forward Backtest Design
8. Evaluation Framework & Benchmarking
9. Critical Pitfalls & Safeguards
10. Repository Structure & Integration
11. Role Assignments & Timeline

1. Strategy Architecture Overview

The strategy has three components that operate in sequence at each monthly rebalancing date:

Component 1 — Return Prediction (Person B). An ML model (XGBoost primary, with Lasso and neural network baselines) takes sector-relative stock characteristics as input and outputs a predicted next-month excess return for every stock in the investable universe. The model learns a cross-sectional mapping: given these feature values, expect this return. It is time-blind and identity-blind — it has no concept of which stock or which date a row came from.

Component 2 — Regime Detection (Person C). A Gaussian Mixture Model fitted to macro-financial features (VIX level, yield curve slope, credit spreads, market volatility, correlation structure) classifies the current market environment into regimes: calm, moderate, or turbulent. This classification determines risk parameters for the portfolio.

Component 3 — Portfolio Construction & Risk Management (Person A + C). Stock predictions are converted into a sector-neutral long-short portfolio. The regime classification modulates three risk levers: overall position sizing (leverage), the number of stocks in each leg (breadth), and the entry threshold (conviction filter). The backtest engine executes this monthly, tracks transaction costs, and computes performance metrics.

Monthly Data Flow

Features (A) → ML Model (B) → Stock Scores
Macro Data (A) → GMM (C) → Regime Label + Risk Parameters
Stock Scores + Risk Parameters → Portfolio Construction → Position Weights
Position Weights → Backtest Engine (A) → P&L, Metrics

2. Why Long-Short with Regime Risk Management

2.1 Why a Fixed Long-Short Structure

A long-short portfolio goes long stocks the model predicts will outperform and shorts stocks predicted to underperform. By construction, the long and short legs approximately cancel out market exposure, meaning the portfolio's returns come from the **spread** between winners and losers, not from the market going up or down. This has a critical advantage: the strategy can generate positive returns in both bull and bear markets, as long as the model correctly identifies relative winners.

2.2 Why Not an Adaptive Portfolio Structure

We initially considered a design where the regime model would fundamentally change the portfolio's character — switching between directional exposure in trending markets and market-neutral positioning in uncertain markets. This approach has two fatal problems:

Transaction cost explosion. The model predicts returns monthly. If the regime flips, you would restructure the entire portfolio in a single month — going from net long +0.8 to net neutral 0.0. That is enormous turnover, and with monthly rebalancing you cannot spread it gradually.

Logical contradiction. If you are running a market-neutral portfolio, knowing the market direction is irrelevant — your returns come from the spread. The regime model would be solving a problem the portfolio does not have. Conversely, if you want directional exposure, you do not need the complexity of a cross-sectional ML model — a simple market-timing signal would suffice.

2.3 The Coherent Design: Regime as Risk Manager

The regime model's job is not to change *what kind* of risk the portfolio takes, but to manage *how much* risk. In calm, low-volatility regimes, cross-sectional predictions are more reliable (correlations are lower, idiosyncratic signals are stronger), so we run the strategy at full size. In high-volatility or crisis regimes, correlations spike, everything moves together, and cross-sectional predictions become noisy — so we scale down: smaller positions, fewer stocks, higher conviction thresholds.

Regime	Leverage	Stocks per Leg	Entry Threshold	Rationale
Calm	1.0x (full)	50 long / 50 short	Standard (top/bottom decile)	Predictions reliable; maximize breadth
Moderate	0.7x	35 long / 35 short	Top/bottom 7%	Slightly elevated noise; trim edges
Turbulent	0.4x	20 long / 20 short	Top/bottom 4% (highest conviction only)	Correlations spike; only trust strongest signals

Key benefit: This approach avoids the transaction cost explosion because we never fundamentally change the portfolio's character. We are just dialing risk up or down within the same structure. Turnover from regime switches is modest — we

trim or add positions at the edges, not flip the entire book.

3. The ML Prediction Engine (Person B)

3.1 What the Model Learns

The model learns a timeless, cross-sectional mapping: given a vector of stock characteristics at date t , predict the stock's excess return over the next month. Each row in the training data is a snapshot (stock, date) — the model processes rows independently. It has no concept of which stock, which date, or which sector a row came from. It is a pure pattern matcher on feature vectors.

Because the model is time-blind, it cannot learn market flow or regime dynamics on its own. If regime information matters (and it does — that is why Person C exists), it must be engineered explicitly into features (e.g., current VIX level as a feature) or handled outside the model (which is our approach via the regime overlay).

3.2 Feature Engineering: The Three-Layer Stack

This is the de-facto standard in quantitative equity and is vastly more effective than any single sampling-based fix for sector imbalance.

Layer 1: Sector-relative features.

Replace raw features with within-sector ranks or z-scores. A utility stock ranked 0.9 is in the top 10% of utilities; a tech stock ranked 0.9 is in the top 10% of tech. Same scale, cross-sector comparable. This removes the sector imbalance problem by itself in most cases.

```
panel[col + '_rel'] = panel.groupby(['date', 'sector'])[col].rank(pct=True)
```

Layer 2: Sector-relative targets.

Predict excess return over sector mean instead of raw return. The learning objective becomes sector-neutral by construction.

```
panel['y_rel'] = panel.groupby(['date', 'sector'])['y'].transform(lambda x: x - x.mean())
```

Layer 3: Sector-neutral portfolio construction.

Even with a biased model, force balance at the trading stage: long/short the top/bottom within each sector, not globally. This prevents sector concentration regardless of model bias.

```
Longs: top k per sector x 11 sectors = 55 stocks
Shorts: bottom k per sector x 11 sectors = 55 stocks
```

3.3 Why Sector-Neutral Matters for Long-Short

Without sector-neutral construction, a long-short strategy on the S&P 500 suffers from severe sector concentration. The model is more confident on tech (the largest sector, ~70-80 stocks, ~30% weight), so the top/bottom 50 picks end up dominated by tech. The portfolio becomes a hidden long-tech/short-tech bet instead of a genuine cross-sectional spread. Three failure modes arise: sector concentration in picks, missed opportunities in small sectors, and overconfident extremes on small-sector stocks.

3.4 Model Selection

Model	Role	Key Hyperparameters
Lasso (OLS + L1)	Linear baseline	Regularization strength (alpha)
XGBoost (GBRT)	Primary model	Depth L, learning rate nu, number of trees B
Feedforward NN	Nonlinear baseline	Hidden layers, dropout, learning rate

Following GKK (2020), tree-based methods and neural nets tend to beat linear models. XGBoost is the primary model because it handles nonlinearities and interactions well, trains fast, and is robust to hyperparameter choices.

3.5 Lag Features and Dynamics

Since the random forest processes rows independently and cannot see evolution, temporal patterns must be pre-digested into scalar features. Two approaches:

Lag features: Add columns x_{t-1} , x_{t-2} for each original feature. Each row now contains a short trajectory, and the model can learn patterns like "rising momentum leads to positive next return."

Dynamics features: Engineered columns like change in momentum over 3 months, volatility acceleration, distance from 52-week high.

4. The Data Pipeline (Person A)

4.1 Universe Construction: Pull Broadly, Filter Strictly

The database must contain every stock that was ever a member of the S&P 500 at any point between 2005 and 2024 — approximately 1,000 unique tickers. Using only today's 500 stocks introduces survivorship bias: your 2008 universe silently excludes Lehman Brothers, Bear Stearns, Washington Mutual, and every other company that went bankrupt, while including Tesla, Meta, and other giants that were not yet public. This inflates the Sharpe ratio by 0.3–0.8 for free and the strategy would collapse in production.

Rule: Pull broadly, filter strictly. Pull every ticker that ever touched the index (the database). At each rebalance, filter to only tickers in the index on that specific date (the investable universe).

4.2 Point-in-Time Universe Filter

A ticker is investable on rebalance date t if and only if all four conditions hold:

1. It was an S&P 500 constituent on date t (from `constituents.parquet`).
2. It has a non-NaN price on t (still actively trading).
3. It has non-NaN values for every factor the model uses (sufficient history).
4. The model produced a non-NaN prediction for it.

If any condition fails, the stock is skipped that month. No imputation. No substitution.

4.3 Missing Data: NaN Is Information

Prices: NaN before IPO, NaN after delisting. Never backfill, never interpolate. The backtest must realize the final return on delisted stocks — a Lehman position held through September 2008 realizes a -99% return. The strategy cannot magically step out before delisting.

Fundamentals: Forward-fill within each stock over time (the one place forward-filling is correct). Never cross-sectionally fill (never replace a missing value with the sector average). Apply a **45-day reporting lag** — a Q1 report (quarter-end March 31) is not publicly available until mid-May. Enforce `fundamentals.shift(45_days)` inside `data_loader.py` so no downstream code can violate it.

Look-ahead bias trap. Using fundamentals on their quarter-end date instead of their report date is the single most common look-ahead bug in student projects. It quietly inflates Sharpe by 0.2–0.5. Enforce the 45-day lag inside `data_loader.py`.

4.4 Raw Data Bundle

Source	Fields	Frequency
Price data	Adjusted close, volume, high, low, open	Daily

Fundamentals	Total assets, total equity, total liabilities, net income, revenue, operating income, CF from ops, shares outstanding, dividends paid	Quarterly
Market metadata	GICS sector, GICS industry, market cap	Monthly
Macro (for Person C)	VIX, DGS10, DGS2, DBAA, DAAA, DFF	Daily (from FRED)

4.5 File Layout

File	Contents	Shape
data/prices.parquet	Daily adjusted close, wide format	~5000 days x 1000 tickers
data/fundamentals.parquet	Quarterly fundamentals, long format	(date, ticker) MultiIndex
data/constituents.parquet	Month-end index membership	(date, ticker, in_index)
data/macro.parquet	Daily macro series	~5000 days x 6 columns

4.6 Mandatory Sanity Checks

Before Person B trains a single model, Person A must run three tests. All must pass; otherwise every number the project produces is wrong.

1. Random-prediction test. Feed random predictions into the backtest. Sharpe must be approximately zero. If materially above zero, there is a bug — likely look-ahead.

2. Oracle test. Feed actual future returns as predictions. Sharpe must be very large (5+). Confirms the pipeline correctly signs trades.

3. Uniform-prediction test. Predict the same value for every stock every date. Strategy return must be approximately zero. Confirms no unexpected dependencies on prediction magnitudes.

Data sanity: Plot universe size over time (should hover ~500). Plot return distributions (symmetric around 0–1%). Spot-check 5 famous events: Lehman (Sept 2008), COVID crash (Mar 2020), Tesla inclusion (Dec 2020), GME squeeze (Jan 2021), a known acquisition.

5. The Regime Overlay (Person C)

5.1 Purpose: Risk Management, Not Direction

The regime model does not predict market direction. It classifies the current market environment to determine how much risk the long-short strategy should take. The insight is that cross-sectional return predictions are more reliable when volatility is low and correlations are moderate (calm markets) than when volatility spikes and correlations go to one (crisis markets). The regime model detects these conditions and adjusts risk parameters accordingly.

5.2 GMM Input Features

Candidate features for the Gaussian Mixture Model (all available from FRED/market data):

1. **VIX level** — implied volatility of S&P 500 options (fear gauge).
2. **Yield curve slope** (DGS10 - DGS2) — recession predictor.
3. **Credit spread** (DBAA - DAAA) — risk appetite in corporate bond markets.
4. **Realized volatility** — trailing 21-day standard deviation of S&P 500 returns.
5. **Average pairwise correlation** — how much stocks move together (higher = less room for stock-picking).

5.3 The Three Risk Levers

The regime label maps to three adjustable parameters that control portfolio risk without changing the portfolio's fundamental character:

Lever 1 — Position sizing (leverage). In calm regimes, invest at full notional. In turbulent regimes, scale all positions down by a multiplier (e.g., 0.4x). The remaining cash sits in the risk-free asset.

Lever 2 — Number of stocks per leg (breadth). In calm regimes, hold 50 longs and 50 shorts (broad diversification). In turbulent regimes, hold only 20 longs and 20 shorts — the highest-conviction picks. Noisier predictions at the margin are cut.

Lever 3 — Entry threshold (conviction filter). In calm regimes, enter any stock in the top/bottom decile within its sector. In turbulent regimes, only enter the top/bottom 4%. This is a natural complement to breadth reduction.

5.4 Transaction Costs from Regime Switches

Because the portfolio structure stays the same (always long-short, always sector-neutral), regime switches only cause *marginal* turnover — trimming some positions from the edges (going from 50 to 35 stocks per leg) or adding them back. This is qualitatively different from the adaptive design where a regime switch could flip the entire book. Person C must still track regime-switch frequency and the associated incremental turnover in the validation sample. If the GMM flips regimes every month, the trimming costs will erode alpha.

6. Portfolio Construction

6.1 The Monthly Rebalancing Procedure

At each month-end rebalance date t :

1. **Filter universe** using the four conditions from Section 4.2.
2. **Generate predictions** for all stocks in the universe using the ML model.
3. **Rank predictions within each sector** — this is the sector-neutral step.
4. **Query the regime model** for the current regime label.
5. **Look up risk parameters** for that regime (leverage, breadth k , threshold).
6. **Select longs**: top k stocks per sector (by within-sector prediction rank).
7. **Select shorts**: bottom k stocks per sector.
8. **Equal-weight within each leg**, then apply leverage multiplier.
9. **Compute turnover** relative to previous month's positions. Apply transaction costs.

6.2 Dollar Neutrality

The portfolio is dollar-neutral: total long notional equals total short notional. Net market exposure is approximately zero in all regimes. What changes across regimes is the gross exposure (total long + total short) and the number of positions, not the net exposure.

6.3 Transaction Cost Model

Apply a realistic cost per unit of turnover. A standard assumption for S&P 500 stocks is 5–10 basis points per one-way trade (covers bid-ask spread + market impact for liquid large-caps). Total monthly cost = turnover $\times$ cost-per-unit. Track separately: regular rebalancing turnover (from prediction changes) and regime-switch turnover (from breadth/leverage adjustments).

7. Walk-Forward Backtest Design

7.1 The Cardinal Rule

Train on the past; test on the future. Always. No exceptions, ever, for any reason. Training data must contain only information that would have been available at the moment of any test-time prediction. Violating this produces a test score that measures retrofitting, not forecasting.

7.2 The Three-Way Split

Following GKX (2020), we split the 2005–2024 sample into three non-overlapping periods:

Sample	Period	Purpose
Training	2005-2015	Estimate model parameters
Validation	2016-2018	Tune hyperparameters (L, nu, B for XGBoost; regime thresholds)
Testing	2019-2024	Final out-of-sample evaluation. Data never enters training or tuning.

The walk-forward approach expands the training window at each rebalance: train on 2005–t, validate on t+1 to t+2, predict t+3. The test sample is sacred — it is never used for any decision.

7.3 Purging and Embargoing

Two technical refinements that prevent information leakage at the train/test boundary:

Purge: If the target at time t covers a forward window (e.g., return from t to t+21 days), drop any training row whose label window overlaps the test window. Otherwise the training label literally contains information about the test label.

Embargo: After a test block ending at date T, exclude the next few days from the subsequent training block. Prevents serial-correlation leakage at the boundary.

[train] | [purge] | [test] | [embargo] | [train]

8. Evaluation Framework & Benchmarking

8.1 The Benchmarking Problem for Long-Short Strategies

A well-constructed long-short portfolio has much less market risk than the S&P 500 — that is the whole point. So in a year where the index returns 25%, the strategy might return 8%. That is not a failure — it took far less risk. Conversely, when the index drops 30%, the strategy might return 4%. Comparing raw returns does not capture this asymmetry.

The right way to evaluate a long-short strategy is on risk-adjusted terms. The Sharpe ratio measures return per unit of risk. A strategy with Sharpe 1.5 and 8% annual return is a far better investment engine than the S&P 500 with Sharpe 0.4 and 10% return, because an investor can lever the strategy to match or exceed the index return while carrying less risk.

8.2 The Three-Level Evaluation

Level 1 — Prediction Quality (does the ML model work?).

Out-of-sample R^2 benchmarked against zero (the GKX standard). If the model cannot beat a naive forecast of zero for individual stock returns, it has no useful information. Also compute R^2 benchmarked against the S&P 500 average return — this measures pure cross-sectional discrimination ability, which is what matters for a long-short portfolio.

Level 2 — Strategy Performance (does the long-short portfolio generate alpha?).

Primary metric: **Sharpe ratio** of the long-short portfolio (net of transaction costs). Compare against the S&P 500 Sharpe ratio over the same period. Additional metrics: annualized return, annualized volatility, maximum drawdown, Calmar ratio, monthly hit rate, average turnover. For a volatility-normalized comparison: lever the strategy to match the index's volatility, then compare raw returns.

Level 3 — Regime Overlay Value (does the GMM add value?).

Compare the full strategy (with regime-based risk scaling) against the same long-short strategy run at constant leverage with no regime overlay. The regime model should improve the Sharpe ratio (primarily by reducing drawdowns during turbulent periods) without proportionally sacrificing return.

8.3 Confidence Intervals

Use block bootstrap over time for confidence intervals on Sharpe ratio, hit rate, and information coefficient. This preserves volatility clustering in the uncertainty estimate. Typical block length for monthly equity data: 6–12 months.

8.4 Diebold-Mariano Test for Model Comparison

To compare prediction quality between models (e.g., XGBoost vs. Lasso), use the adapted Diebold-Mariano test from GKX: average prediction error differences across stocks at each date to form a single time series, then test whether the mean differs from zero using Newey-West standard errors. This avoids the cross-sectional dependence problem.

9. Critical Pitfalls & Safeguards

Pitfall	Why It Breaks	Remedy
Survivorship bias	Only surviving stocks in universe; hides losers	Pull all historical S&P 500 constituents (~1000 tickers)
Look-ahead in fundamentals	Using data before publicly available	45-day reporting lag enforced in data_loader.py
Random shuffled train/test	Future-to-past leakage; correlated neighbors	Walk-forward split, always
Overlapping label windows	Test label literally in training data	Purge overlapping rows
Adjacent-day leakage	Serial correlation at boundary	Embargo period after test blocks
Tech concentration in picks	Model more confident on large sectors	Sector-relative features + sector-neutral portfolio
Imputing missing data	Creates synthetic signal that does not exist	NaN stays NaN; stock drops out that month
Using RF OOB as primary metric	Cross-sectional correlation makes OOB optimistic	Walk-forward test error as primary metric
Backfilling prices for delisted stocks	Hides bankruptcy losses	Realize final return; then NaN
Regime switches causing high turnover	Transaction costs erode alpha	Only adjust breadth/leverage, not portfolio structure

10. Repository Structure & Integration

10.1 File Structure

```
ml-factor-investing-2026/
|-- data/                # .gitignored; generated by download scripts
|   |-- prices.parquet
|   |-- fundamentals.parquet
|   |-- constituents.parquet
|   |-- macro.parquet
|-- src/
|   |-- __init__.py
|   |-- data_loader.py    # Person A: data ingestion + 45-day lag
|   |-- factors.py        # Person B: factor/feature construction
|   |-- models.py         # Person B: ML models (Lasso, XGBoost, NN)
|   |-- regime.py         # Person C: GMM regime detection
|   |-- portfolio.py      # Person C: regime-conditional risk parameters
|   |-- backtest.py       # Person A: walk-forward backtest engine
|   |-- metrics.py        # Person A: Sharpe, R2, drawdown, DM test
|-- notebooks/           # Exploration only; no production code
|-- results/             # Backtest outputs, plots
|-- report/              # Final report
|-- DECISIONS.md         # Every design decision logged
|-- requirements.txt
```

10.2 Module Interfaces (Agree on Day 1)

The integration between workstreams is critical. Agree on exact function signatures on Day 1:

```
# data_loader.py (Person A)
def load_prices() -> pd.DataFrame: ...
def load_fundamentals() -> pd.DataFrame: ... # 45-day lag already applied
def load_constituents() -> pd.DataFrame: ...
def load_macro() -> pd.DataFrame: ...

# models.py (Person B)
def predict(features: pd.DataFrame, model) -> pd.Series: ... # returns scores

# regime.py (Person C)
def detect_regime(macro: pd.DataFrame, date) -> str: ... #
"calm"/"moderate"/"turbulent"

# portfolio.py (Person C)
def get_risk_params(regime: str) -> dict: ... # {leverage, k_per_sector, threshold}
def construct_portfolio(scores, regime_params, universe) -> pd.Series: ... # weights

# backtest.py (Person A)
def run_backtest(predictions, regime_labels, risk_params, prices, costs) -> dict: ...
```

11. Role Assignments & Timeline

Person	Owns	Key Deliverables
A — Data & Backtest	data_loader.py, backtest.py, metrics.py	Clean data pipeline with survivorship-free universe, 45-day lag, sanity checks passed. Walk-forward backtest engine with transaction costs. Performance metrics vs. S&P 500.
B — Alpha Model	factors.py, models.py	Sector-relative features + sector-relative targets. Lasso baseline, XGBoost primary, NN secondary. Out-of-sample R^2 (both benchmarks). Diebold-Mariano model comparison.
C — Regime & Risk	regime.py, portfolio.py	GMM fitted to macro features. Three-regime classification. Risk parameter mapping table. Sector-neutral portfolio construction with regime-conditional breadth/leverage. Evidence that overlay improves Sharpe vs. constant-leverage baseline.

5-Week Timeline

Week	Focus	Milestones
1	Setup & Data	Repo, environments, data pull complete, sanity checks pass, interfaces agreed
2	Features & Regime	Factor construction done (B), GMM fitted (C), backtest skeleton working (A)
3	Models & Integration	All three models trained (B), regime overlay integrated (C), end-to-end backtest runs (A)
4	Tuning & Evaluation	Hyperparameter tuning on validation set, three-level evaluation complete, regime value tested
5	Report & Polish	Final report, plots, code cleanup, presentation preparation

A clean pipeline, honest evaluation, and modest ambition beat a complex strategy built on leaky data.